

Histórico de Alterações

Sessão de Desenvolvimento — 08/06/2026

Disciplina: Programação II — 2025

Projeto: Assistente de Compras Inteligente

Data: 08/06/2026

1. Resumo das Alterações

Este documento registra todas as alterações realizadas durante a sessão de desenvolvimento de 08/06/2026. As mudanças abrangem correções de bugs, melhorias de performance, novas funcionalidades, expansão do catálogo e atualização dos testes.

Categoria	Qtd. de mudanças
Correções de bugs	4
Melhorias de performance	2
Novas funcionalidades	7
Expansão do catálogo	3
Organização e limpeza	5
Atualizações nos testes	4

2. Correções de Bugs

BUG CRÍTICO

Pilha não inicializava — `__init__` sem underscores duplos

Arquivo: `estruturas/pilha.py`

Problema: o método construtor estava definido como `"def init()"` em vez de `"def __init__()"`. O Python nunca chamava o construtor, então `self._dados` nunca era criado. Qualquer chamada a `push()`, `pop()` ou `historico()` lançava `AttributeError`.

Solução: corrigido para `"def __init__(self, capacidade: int = 50)"`.

BUG CRÍTICO

Servidor não respondia à rota `/buscar` — frontend não funcionava

Arquivo: `servidor.py`

Problema: o servidor usava `SimpleHTTPRequestHandler` puro, que só serve arquivos estáticos. O frontend fazia `fetch("/buscar?q=...")` e recebia 404.

Solução: classe `Handler` estendida com método `do_GET()` que intercepta `/buscar` e `/voltar`, retornando JSON com os resultados.

BUG

Busca por termos sem relação retornava todo o catálogo

Arquivo: `assistente.py`

Problema: quando o NLP não reconhecia categoria nem preço (ex: `"cuscuz"`), o método `buscar()` caía no `else: catalogo.listar_todos()`, retornando todos os 49 produtos. Além disso, `_recomendar_por_historico()` rodava e adicionava mais 2 itens baseados no histórico.

Solução: retorno antecipado quando não há categoria, preço nem marca.

BUG

Adjetivos no feminino não reconhecidos pelo NLP

Arquivo: `nucleo/nlp_parser.py`

Problema: `"televisão barata"` não filtrava por preço porque a lista de keywords tinha `"barato"` (masculino) mas não `"barata"` (feminino). Mesmo problema com `"cara"`, `"caras"`, `"baratas"`, `"econômica"`.

Solução: adicionadas todas as formas: `barata`, `baratas`, `econômica`, `cara`, `caras`, `média`.

3. Melhorias de Performance

PERFORMANCE

FilaSimples: `list.pop(0)` → `deque.popleft()`

Arquivo: `estruturas/fila_simples.py`

Problema: `list.pop(0)` em Python é $O(n)$ — desloca todos os elementos.

Solução: substituído por `collections.deque` com `popleft()`, que é $O(1)$. Para filas com volume alto de requisições, a diferença é significativa.

PERFORMANCE

Pilha: `list + pop(0)` manual → `deque(maxlen=capacidade)`

Arquivo: `estruturas/pilha.py`

Problema: quando a pilha atingia a capacidade, fazia `pop(0)` para remover o item mais antigo — também $O(n)$.

Solução: `deque` com `maxlen` faz remoção automática do elemento mais antigo ao atingir o limite, em $O(1)$.

4. Novas Funcionalidades

FEATURE

Reconhecimento de marcas no NLP

Arquivo: `nucleo/nlp_parser.py`

Adicionado dicionário `MARCAS` com 24 fabricantes e seus termos de busca. O campo "marca" é agora extraído pelo `parsear()` e retornado no resultado.

Exemplos: "samsung" → `marca=samsung`; "celular da apple" → `categoria=celular + marca=apple`; "moto" → `marca=motorola`.

FEATURE

Campo marca nos produtos do catálogo

Arquivo: `dados/catalogo.py`

Todos os 49 produtos receberam o campo "marca" com o fabricante normalizado (ex: "apple", "samsung", "motorola"). Isso permite filtrar os resultados exatamente pela marca solicitada.

FEATURE

Plurais de categorias reconhecidos

Arquivo: `nucleo/nlp_parser.py`

Adicionadas formas plurais em todas as categorias: "celulares", "notebooks", "tvs", "fones", "tablets", "smartwatches", "consoles", "monitores". Antes, buscas no plural retornavam vazio.

FEATURE

Separação visual: Resultados vs Recomendados

Arquivos: `servidor.py`, `index.html`

O servidor passou a retornar JSON estruturado com dois campos separados: "resultados" (busca direta) e "recomendacoes" (grafo + histórico). O frontend exibe as duas seções com headers distintos, deixando claro para o usuário o que é resultado direto e o que é sugestão.

FEATURE

Botão Voltar na interface web

Arquivos: `servidor.py` (rota `/voltar`), `index.html` (botão + função JS)

Implementado botão "← Voltar" que aparece após a primeira busca. Ao clicar, chama `/voltar` no servidor, que usa o método `voltar()` da Pilha para retornar aos resultados da busca anterior.

FEATURE

Mensagem de não encontrado com sugestões

Arquivo: *index.html*

Quando a busca não retorna resultados, em vez de só "Nenhum produto encontrado", o sistema exibe chips clicáveis com todas as 8 categorias disponíveis, ajudando o usuário a reformular a busca.

FEATURE

Recomendação por histórico exclui categoria atual

Arquivo: *assistente.py*

O método `_recomendar_por_historico()` passou a receber a categoria da busca atual como parâmetro. Ela é excluída da análise de histórico, evitando que produtos da mesma categoria apareçam como "recomendados" quando o usuário já está vendo todos eles nos resultados diretos.

5. Expansão do Catálogo

REFACTOR

Catálogo movido para módulo separado dados/catalogo.py

Antes: a lista de produtos e relações estava inline no método `_popular_catalogo()` em `assistente.py`, tornando o arquivo longo e difícil de manter.

Depois: `dados/catalogo.py` contém `PRODUTOS` e `RELACOES` como constantes. `assistente.py` importa com `"from dados.catalogo import PRODUTOS, RELACOES"`. Para adicionar produtos, basta editar `catalogo.py` sem tocar na lógica.

EXPANSÃO

Catálogo: 11 → 49 produtos em 8 categorias

Categorias adicionadas: `tablet`, `smartwatch`, `console`, `monitor`

Produtos por categoria: `celular(9)`, `notebook(8)`, `tv(6)`, `fone(7)`, `tablet(5)`, `smartwatch(5)`, `console(4)`, `monitor(5)`

Relações no grafo: 7 → 40+ (`celular↔fone`, `celular↔smartwatch`, `notebook↔monitor`, `console↔tv`, `tablet↔fone`)

EXPANSÃO

NLP atualizado para 8 categorias e 24 marcas

Arquivo: `nucleo/nlp_parser.py`

Adicionados keywords para: `tablet`, `smartwatch`, `console`, `monitor`.

Adicionado dicionário `MARCAS` com: `samsung`, `apple`, `motorola`, `xiaomi`, `sony`, `lg`, `dell`, `acer`, `hp`, `asus`, `lenovo`, `positivo`, `jbl`, `beats`, `edifier`, `tcl`, `philips`, `nintendo`, `microsoft`, `garmin`, `amazfit`, `aoc`, `multilaser`, `realme`.

6. Organização e Limpeza de Código

LIMPEZA

Arquivos de código morto removidos

Removidos: `estruturas/none.py` (classe `Node` incompleta sem uso) e `main.py` (rascunho de testes com lista crua). Nenhum dos dois era importado em lugar algum do projeto.

LIMPEZA

Bloco `__main__` removido de `assistente.py`

O bloco `"if __name__ == '__main__':"` em `assistente.py` continha frases de teste com emojis que causavam problemas em terminais Windows. Foi removido — os testes adequados ficam na pasta `testes/`.

LIMPEZA

Escrita desnecessária de `data.js` removida

`assistente.py` escrevia um arquivo `data.js` a cada inicialização com os dados do catálogo. O frontend havia migrado para usar a API `/buscar` e não lia mais esse arquivo. A escrita foi removida, junto com os imports `json` e `pathlib` que só serviam para isso.

REFACTOR

Servidor: método `_json()` extraído para evitar repetição

Arquivo: `servidor.py`

O código de serializar JSON e enviar a resposta HTTP era duplicado para cada rota. Extraído para o método auxiliar `_json(dados)` que centraliza `Content-Type`, `Content-Length` e `Access-Control-Allow-Origin`.

MELHORIA

Normalização de acentos no NLP

Arquivo: `nucleo/nlp_parser.py`

Adicionado método estático `_norm()` usando `unicodedata.normalize(NFD)`. Remove acentos

antes de comparar keywords. "câmera" = "camera", "televisão" = "televisao", "econômico" = "economico". Todos os keywords do parser foram simplificados para a forma sem acento.

7. Atualizações nos Testes

TESTES

test_fila_simples.py criado do zero

Arquivo: *testes/test_fila_simples.py*

O arquivo existia mas estava vazio (1 linha em branco). Criados 6 testes: *enqueue/dequeue*, *ordem FIFO*, *esta_vazia*, *tamanho*, *exceção em fila vazia*, *repr*.

TESTES

Métodos novos adicionados em Pilha

Arquivo: *estruturas/pilha.py*

Adicionados para satisfazer *test_pilha.py*:

- *tamanho()* → *int*
- *categorias_recentes(n)* → *lista de categorias das últimas n buscas*
- *__repr__()* → *representação string*

TESTES

Métodos novos adicionados em HashTable

Arquivo: *estruturas/hash_table.py*

Adicionados para satisfazer *test_hash_table.py*:

- *listar_chaves()* → *lista de todas as chaves*
- *listar_itens()* → *lista de tuplas (chave, valor)*
- *__repr__()* → *representação string*

TESTES

Métodos novos adicionados em GrafoProdutos e ArvoreCategorias

Arquivo: *estruturas/grafos.py*

- *produto_existe(id)* → *bool*
- *listar_relacoes()* → *lista de tuplas (a, b, peso) sem duplicatas*
- *recomendar_por_historico(ids)* → *recomendações agregadas de múltiplos produtos*

Arquivo: *estruturas/arvore.py*

- *buscar_categoria(caminho, filtros=None)* → *suporte a filtros opcionais*
- *categoria_existe(caminho)* → *bool*